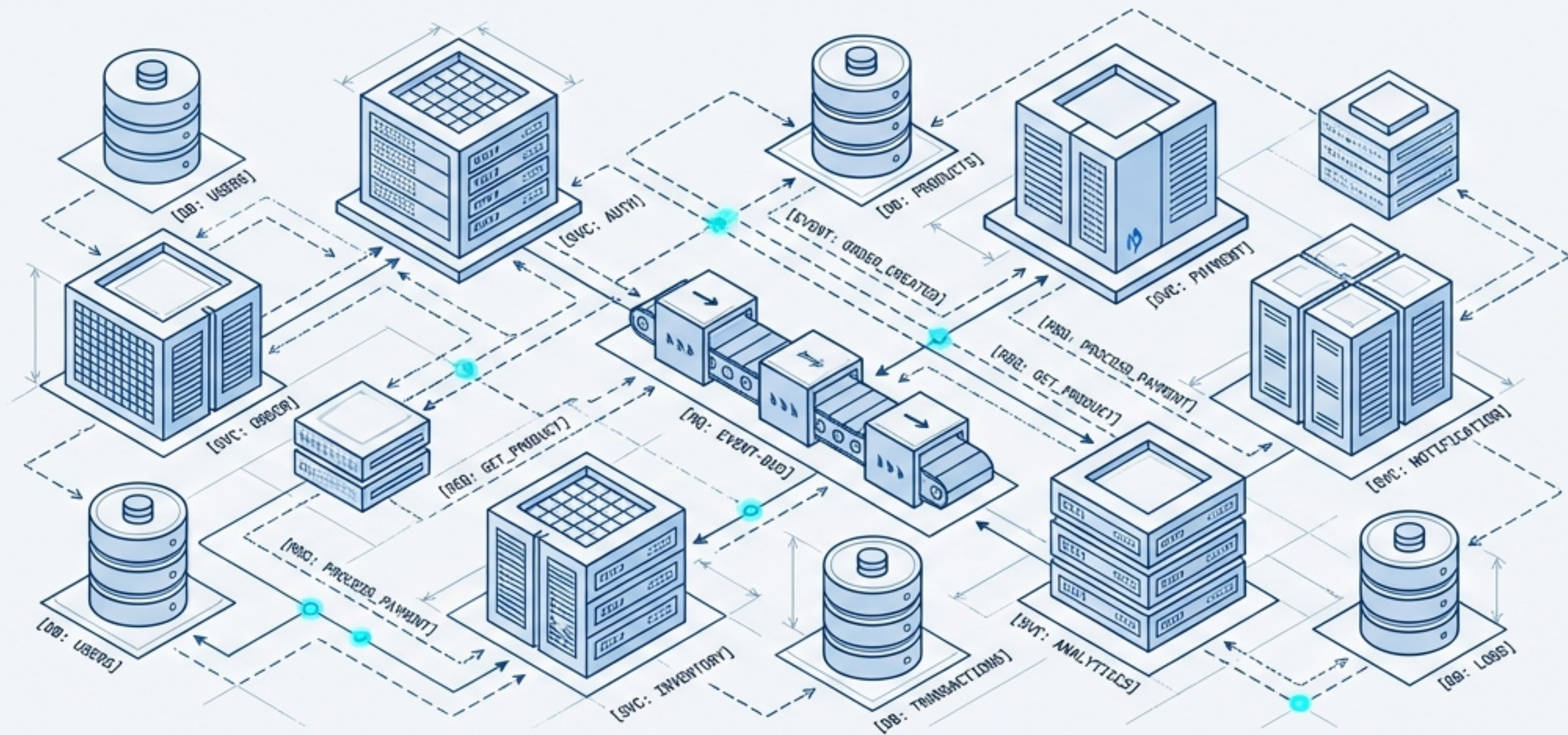




Báo Cáo Chuyên Sâu: Đánh Giá Kiến Trúc & Mã Nguồn Dự Án Microservices

Kiểm toán kỹ thuật toàn diện và lộ trình nâng cấp hệ thống

[STATUS: DRAFT]

[SCOPE: ARCHITECTURE, CODE, SEC, OPS]

[TARGET: V2.0]

Bản Chất Của Kiến Trúc Microservices: Đòn Bẩy & Cái Giá Phải Trả

— Đặc Điểm Cốt Lõi

-  - Tính phi tập trung (Decentralization)
-  - Liên kết lỏng (Loose Coupling)
-  - Gắn kết cao (High Cohesion)
-  - Triển khai độc lập



— Lợi Thế Chiến Lược

-  - Mở rộng cục bộ (Targeted scalability)
-  - Cô lập lỗi (Fault Isolation)
-  - Phát triển song song giữa nhiều đội ngũ

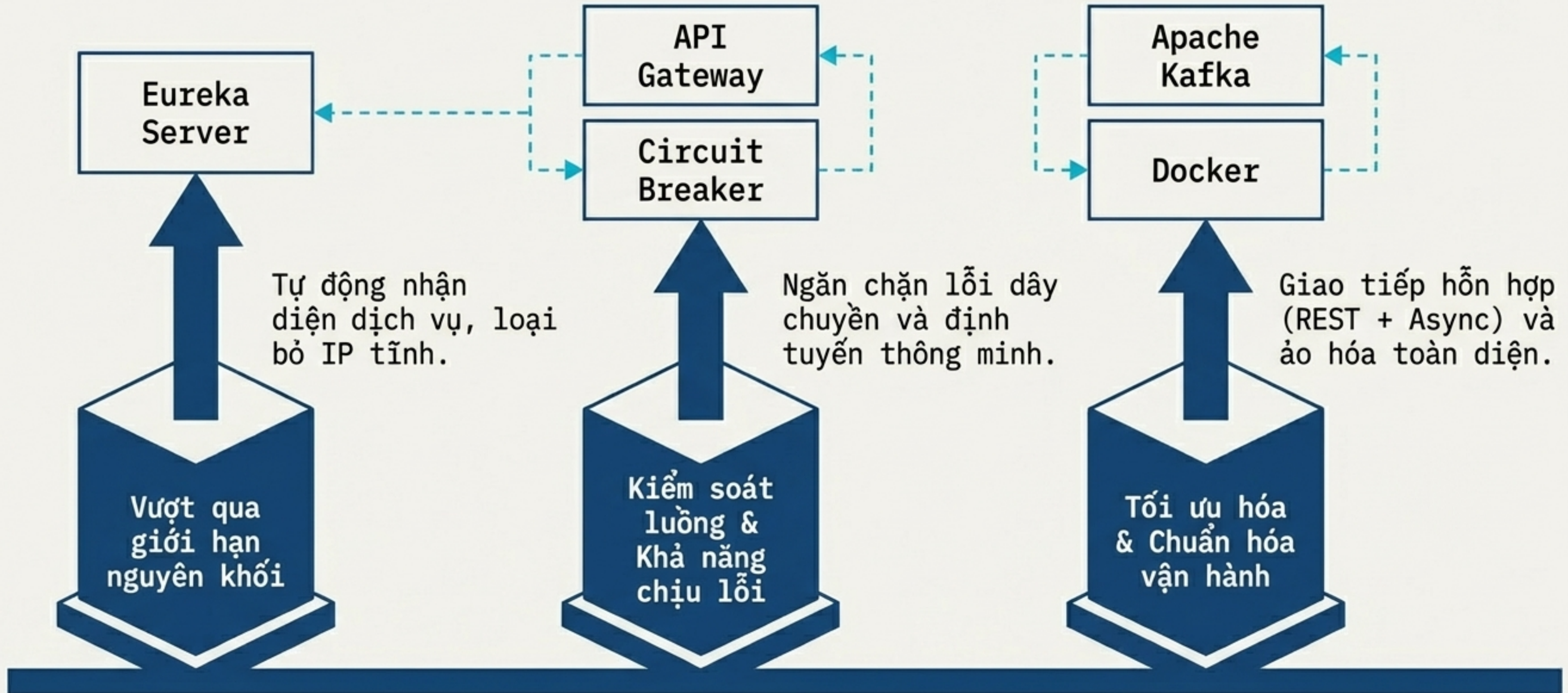
— Điểm Nghẽn Vận Hành

-  - Quản trị phân tán phức tạp
-  - Khó đảm bảo tính nhất quán dữ liệu
-  - Độ trễ mạng (Network Latency)

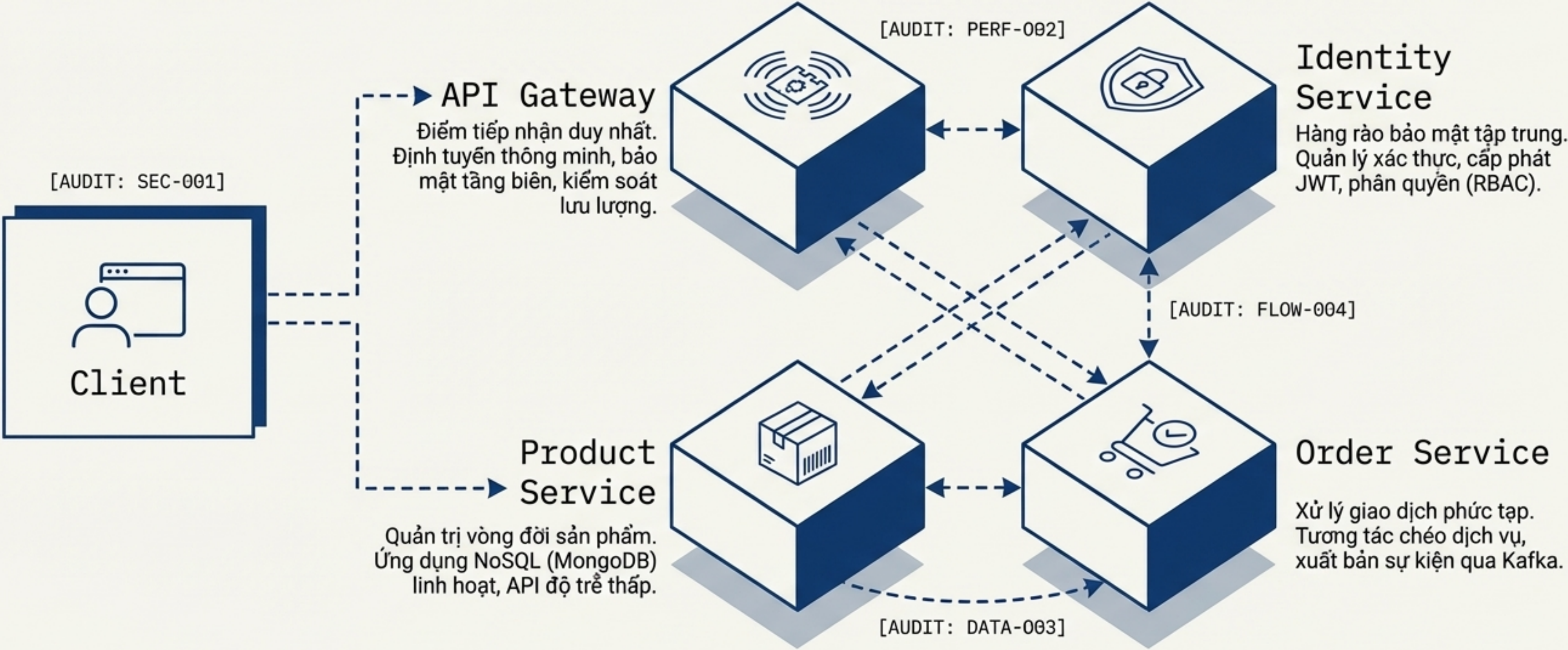
Tiêu Chí Áp Dụng —

-  - Lý tưởng cho hệ thống quy mô lớn, nhiều luồng nghiệp vụ.
-  - Không phù hợp cho dự án nhỏ, ngân sách/nhân sự hạn chế.

Bức Tranh Toàn Cảnh: Mục Tiêu & Định Hướng Dự Án



Hệ Sinh Thái 4 Dịch Vụ Cốt Lõi Được Kiểm Toán



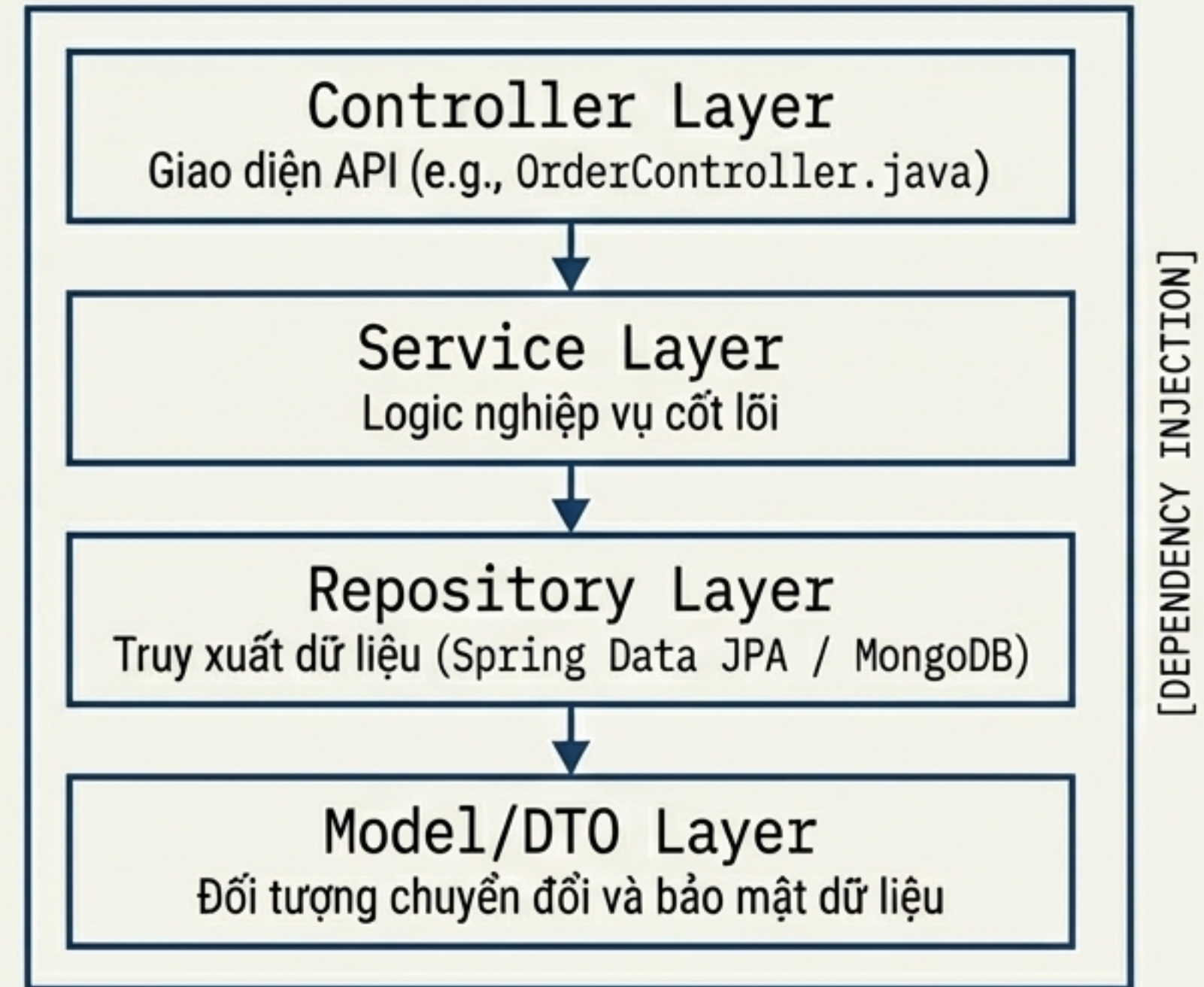
Giải Phẫu Mã Nguồn: Multi-Module & Tầng Kiến Trúc

Cấp Độ Dự Án

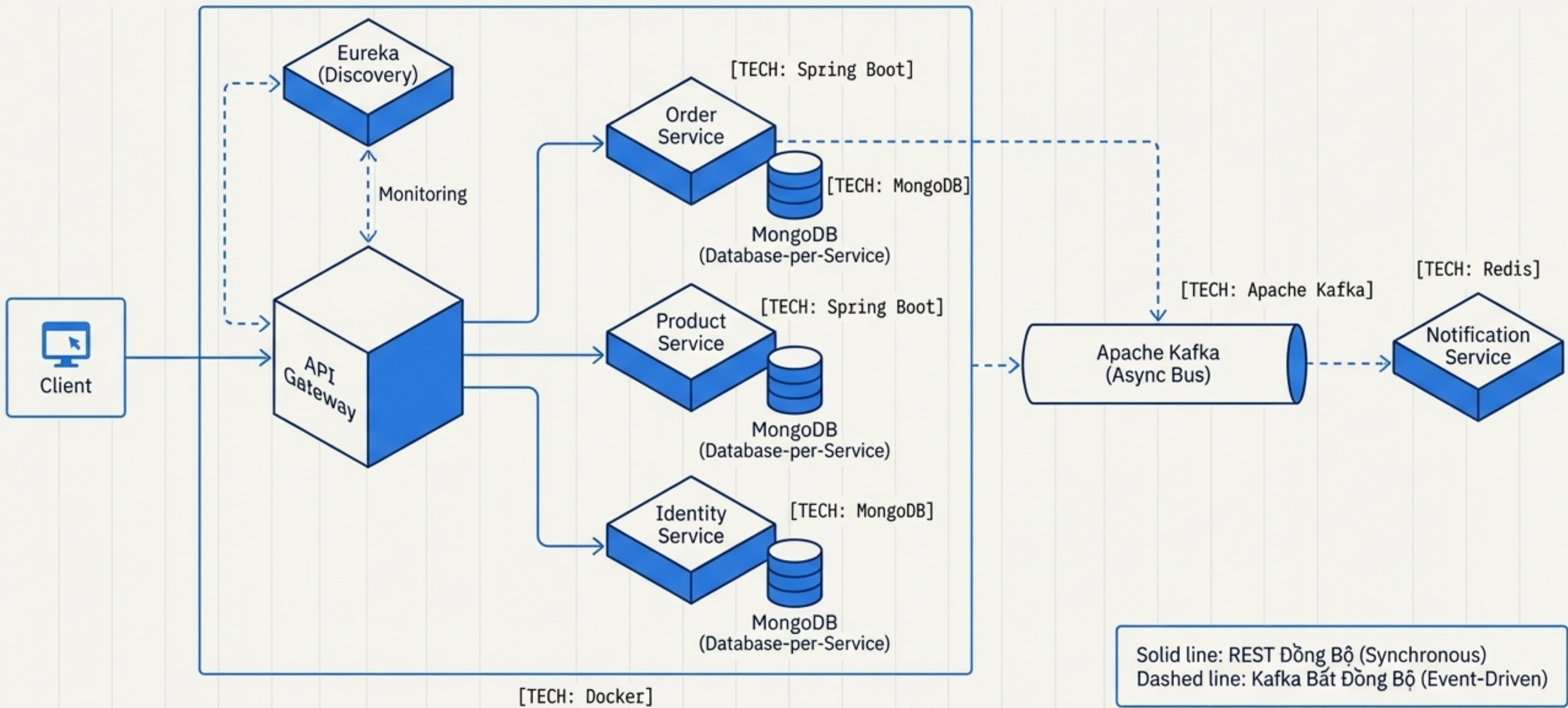
Dự Án Hệ Thống

- microservices_api_demo
 - Các dịch vụ cốt lõi (Gateway, Order, Product)
- ProjectWeb-BE
 - Nền tảng hạ tầng (Eureka, K8s, Docker configs)
- ProjectWeb-main
 - Giao diện người dùng (Frontend)

Kiến Trúc Phân Lớp



Kiến Trúc Tổng Thể & Ngăn Xếp Công Nghệ (Tech Stack)

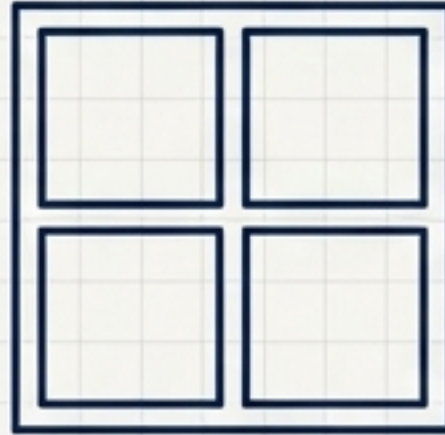


Đánh Giá Kiến Trúc: Nền Tảng Kỹ Thuật Vững Chắc

Domain-Driven Design (DDD)

[PASS]

Ranh giới dịch vụ tường minh. Mỗi dịch vụ xử lý một phạm vi nghiệp vụ duy nhất, giảm thiểu chồng chéo logic.



Database per Service

[PASS]

Tính cô lập dữ liệu tuyệt đối. Sử dụng MongoDB độc lập loại bỏ rủi ro nút thắt cổ chai vật lý.



API Gateway Centralization

[PASS]

Quản lý truy cập tập trung. Đóng vai trò “người gác cổng”, ẩn đi độ phức tạp nội bộ và tối ưu tương tác cho Client.



Asynchronous Offloading

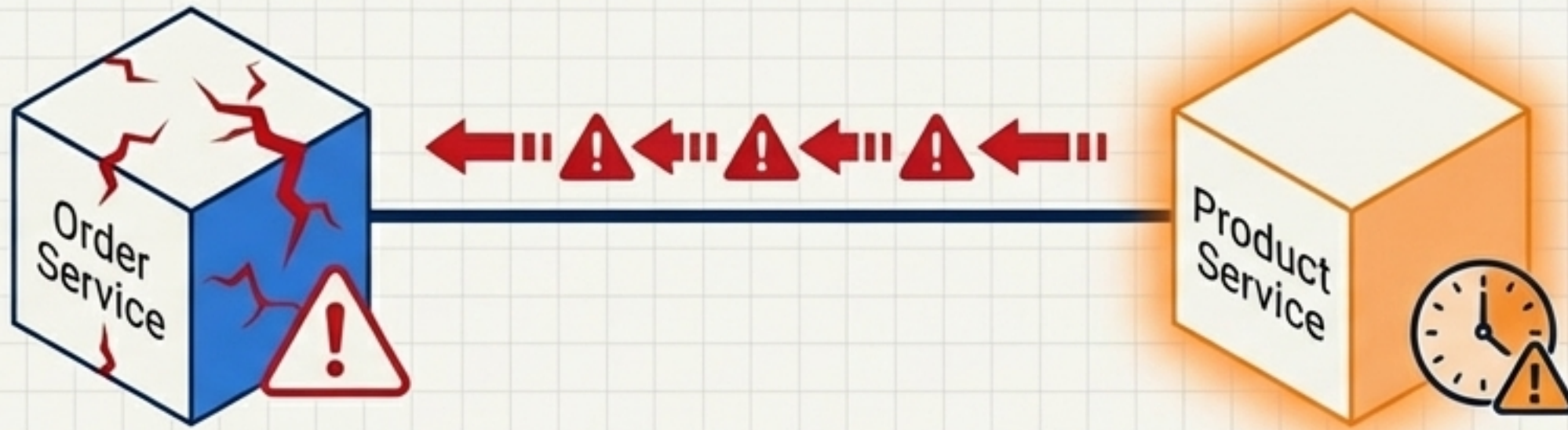
[PASS]

Tận dụng Messaging (Kafka) cho tác vụ nặng, cải thiện thời gian phản hồi và khả năng chịu tải.



Điểm Nghẽn Kiến Trúc: Rủi Ro Chuỗi Đồ Vỡ (Cascading Failures)

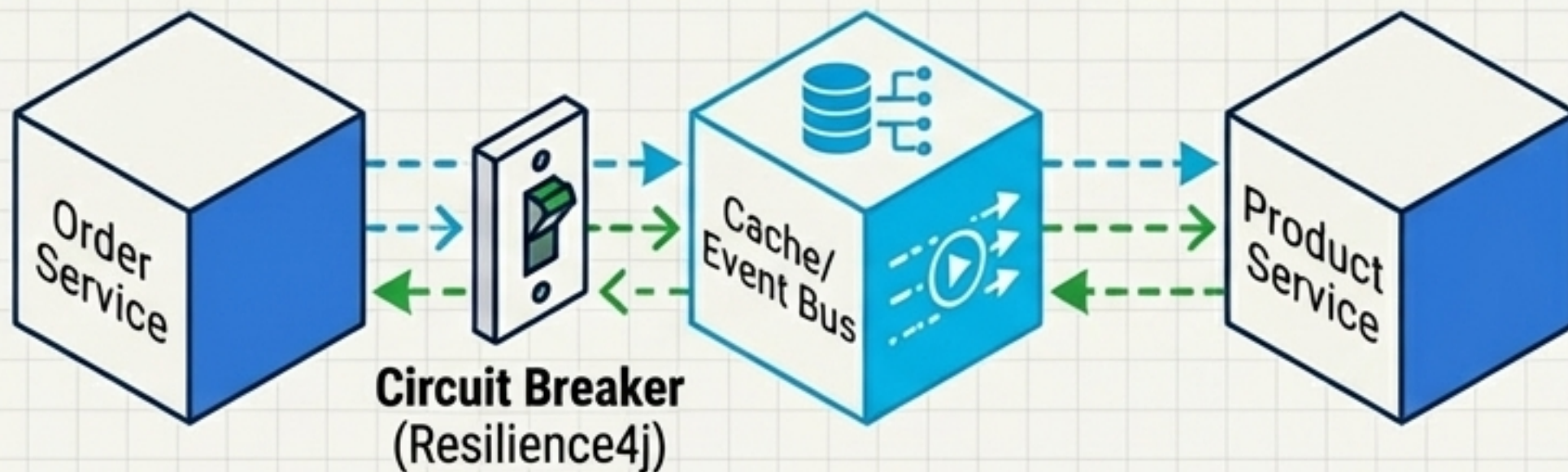
[VULNERABILITY: HIGH COUPLING]



The Problem

Giao tiếp đồng bộ trực tiếp (Order -> Product) gây phụ thuộc cao.
Một độ trễ mạng lớn làm cạn kiệt tài nguyên toàn hệ thống.

[ACTION: REFACTOR]



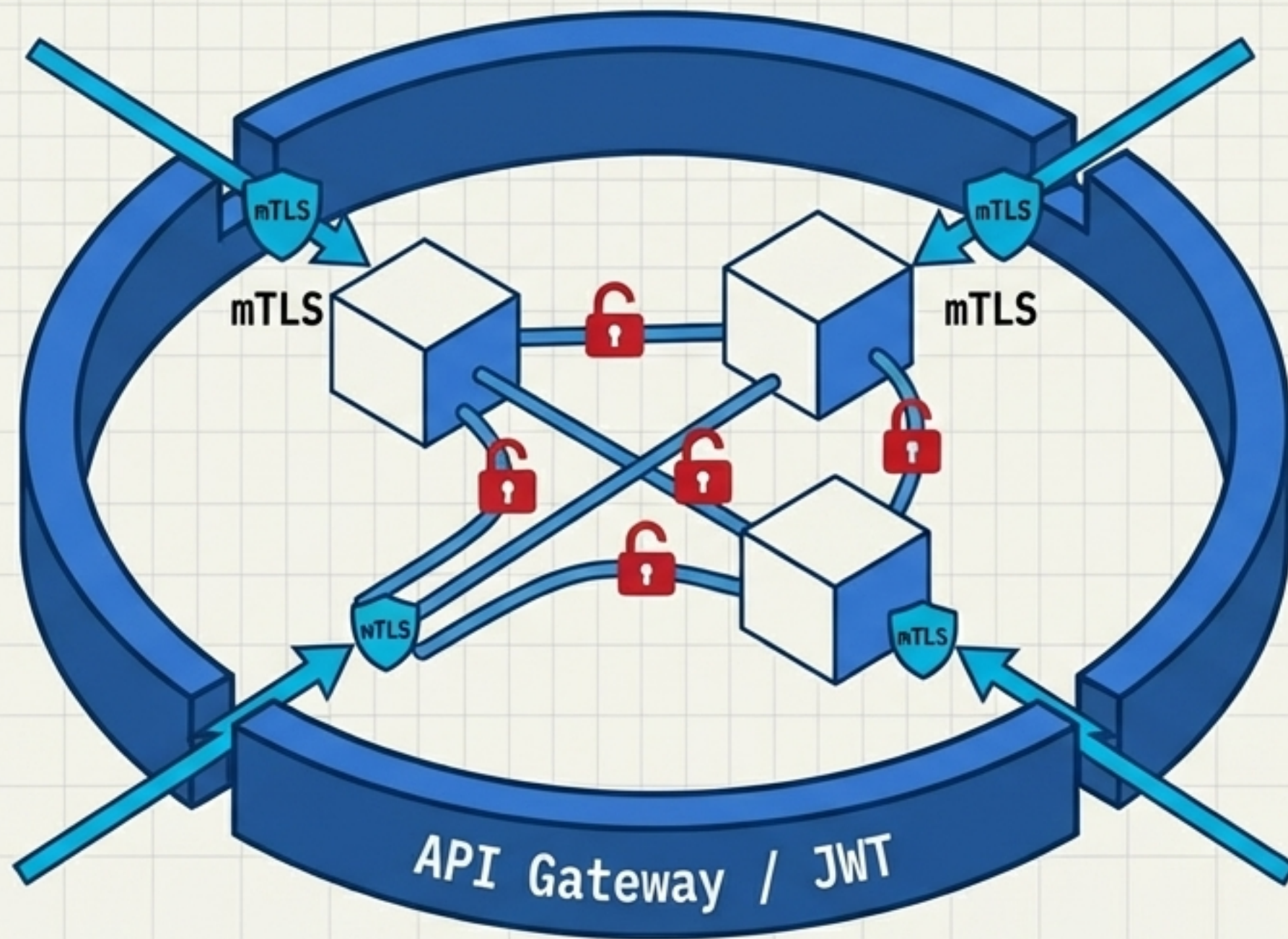
The Solution

- Chuyển đổi sang **Event-Driven & Caching**. Chấp nhận Eventual Consistency.
- Triển khai **Circuit Breaker** (Resilience4j) & Fallback.

Đánh Giá Mã Nguồn: SOLID & Khả Năng Bảo Trì

Hạng Mục	Đánh Giá Hiện Tại	Định Hướng Cải Thiện
Cơ chế DI	[STRENGTH] Tận dụng triệt để Dependency Injection tạo khớp nối lỏng.	Duy trì tiêu chuẩn.
Quy chuẩn Naming	[STRENGTH] Naming Convention tường minh, sát ngữ nghĩa nghiệp vụ.	Duy trì tiêu chuẩn.
Tính lặp lại	[ISSUE] Logic lặp lại ở xử lý lỗi và validation.	[REFACTOR] Xây dựng Shared Libraries & Global Exception Handler.
Nguyên lý SOLID	[ISSUE] Vi phạm SRP. 'God Objects' kiêm nhiệm logic, mapping, và thông báo.	[REFACTOR] Tách biệt Mapper Classes & Services chuyên dụng.

Lỗ Hổng Bảo Mật Nội Bộ: Xây Dựng Hàng Rào Zero Trust



Rủi ro hiện tại

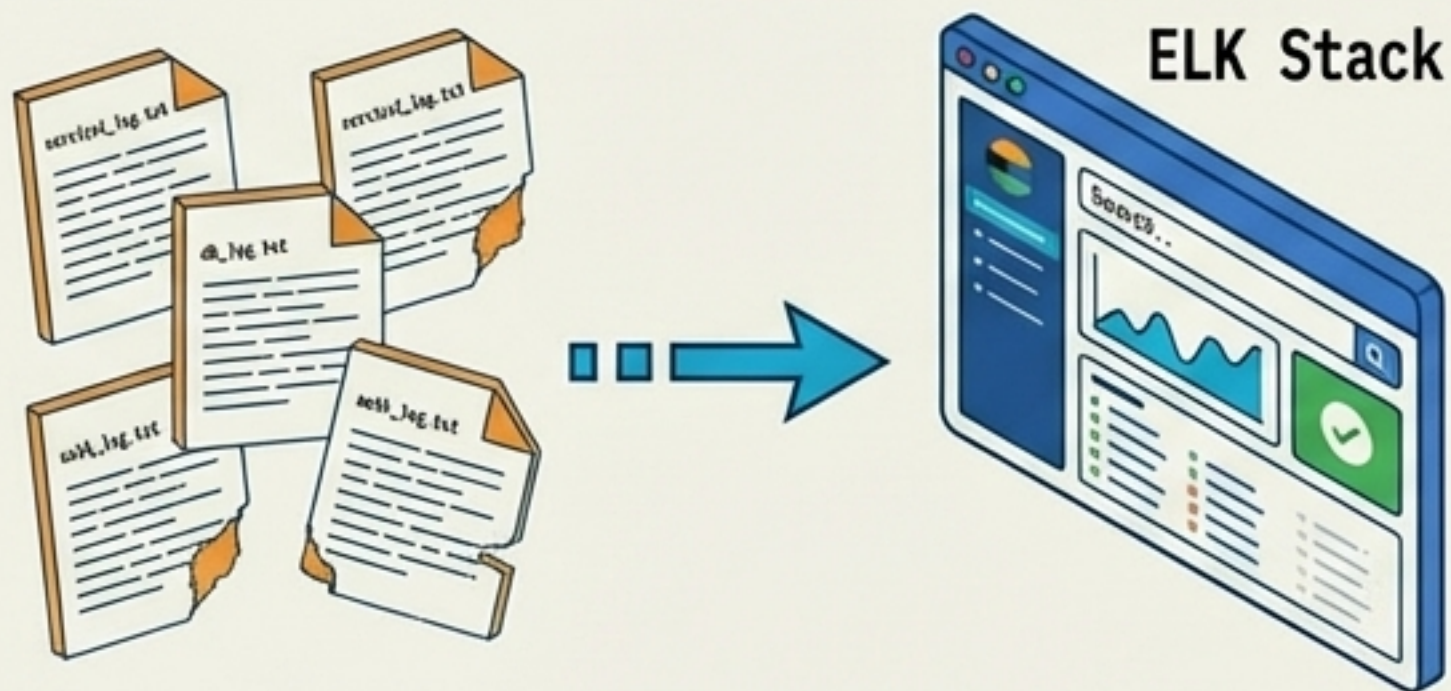
Thiếu xác thực liên dịch vụ. Dịch vụ bị chiếm quyền có thể dễ dàng tấn công chéo hoặc truy cập trái phép vào **MongoDB** nội bộ.

Mục tiêu Zero Trust

- **Distributed JWT Validation:** Mọi request luân chuyển nội bộ đều mang Token định danh.
- **mTLS & API Keys:** Chỉ cho phép giao tiếp giữa các dịch vụ trong **Whitelist**, mã hóa hoàn toàn lưu lượng.

Nút Thắt Vận Hành: Chẩn Đoán Logging & Quản Lý Cấu Hình

Vấn đề Nhật Ký (Logs)

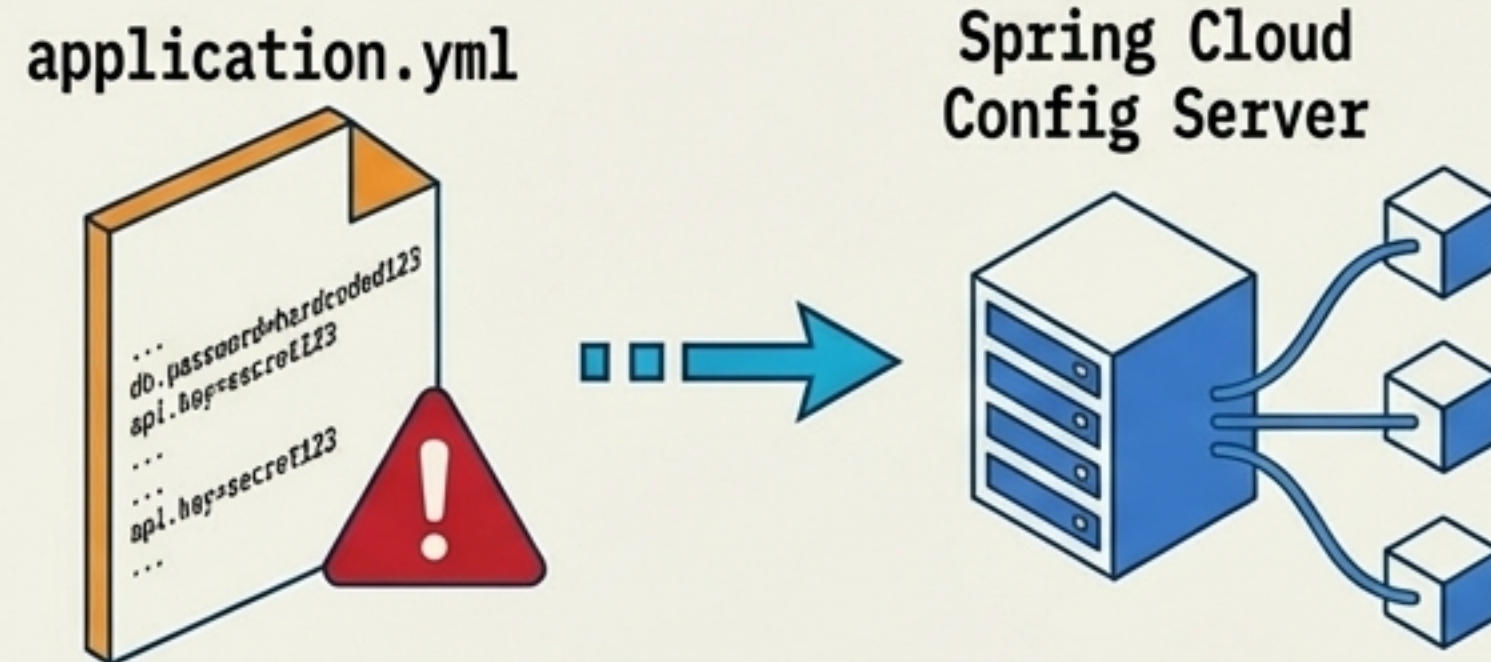


Current: Lưu trữ rời rạc, không thể truy vết lỗi chéo dịch vụ.

[SOLUTION]

Tích hợp **ELK/EFK Stack**. Áp dụng mã định danh **Correlation ID** truyền xuyên suốt hệ thống để truy vết toàn trình.

Vấn đề Cấu Hình (Config)

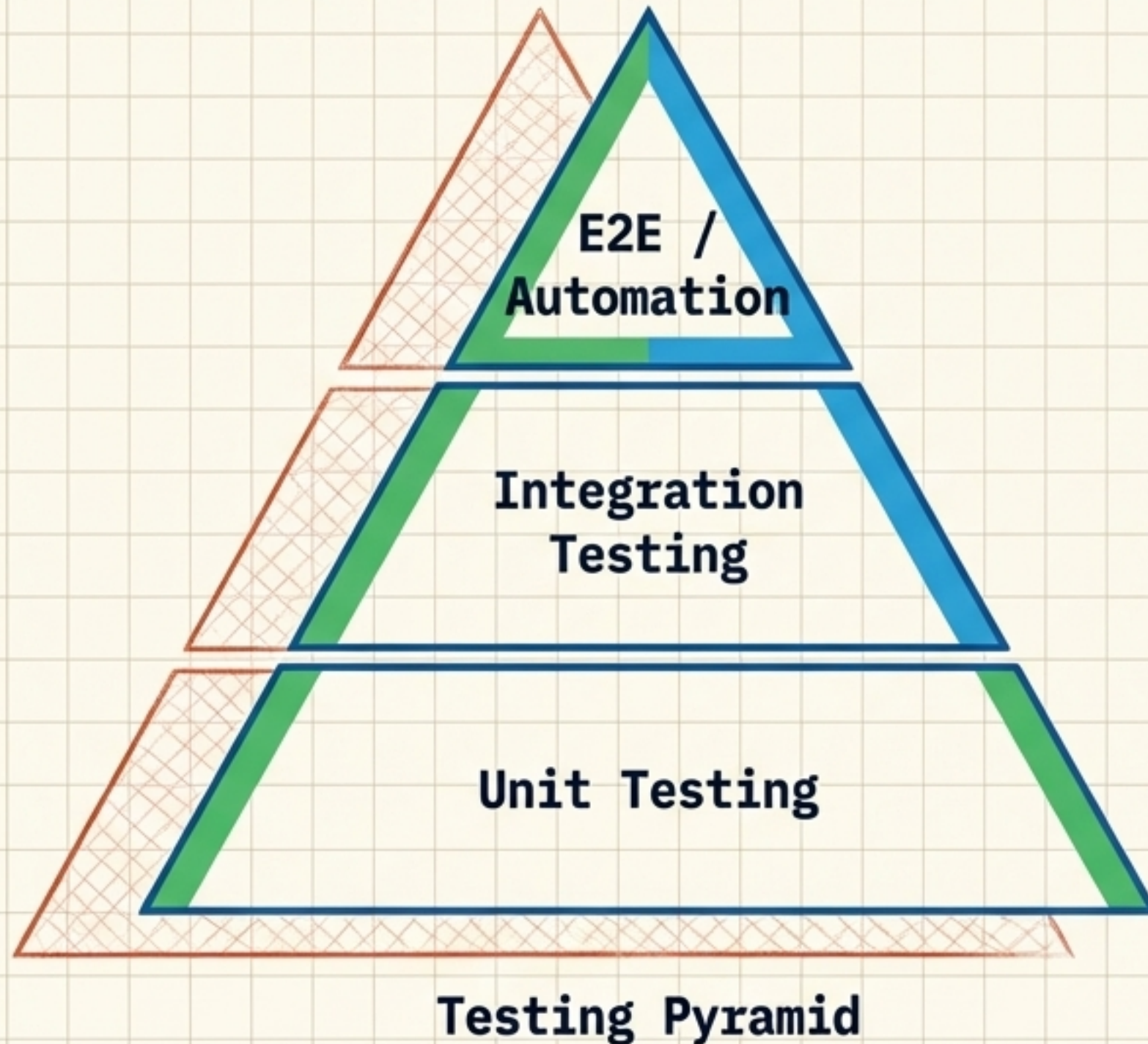


Current: Tham số nhay cảm bị **Hardcoded**, dễ rò rỉ và phải rebuild Image khi đổi môi trường.

[SOLUTION]

Sử dụng **Environment Variables** (Docker) và hướng tới triển khai **Config Server**.

Kỷ Luật Kiểm Thử: Từ Mã Nguồn Đến Tự Động Hóa (CI/CD)



Automation: Tích hợp CI/CD Pipeline. Chặn toàn bộ mã nguồn không vượt qua bài kiểm thử tự động trước khi merge.

Integration Testing: Triển khai Testcontainers để khởi tạo MongoDB/Kafka thực tế trong quá trình chạy test, xóa bỏ các giả lập thiếu tin cậy.

Unit Testing: Nâng độ bao phủ lên 80% tại tầng Service bằng JUnit & Mockito.

Rủi ro hiện tại: Thiếu kiểm thử tích hợp dẫn đến sai lệch dữ liệu liên dịch vụ.

Xóa Bỏ Hộp Đen Vận Hành: Giám Sát & Truy Vết Toàn Diện

[BLINDSPOT: Vận hành mù]

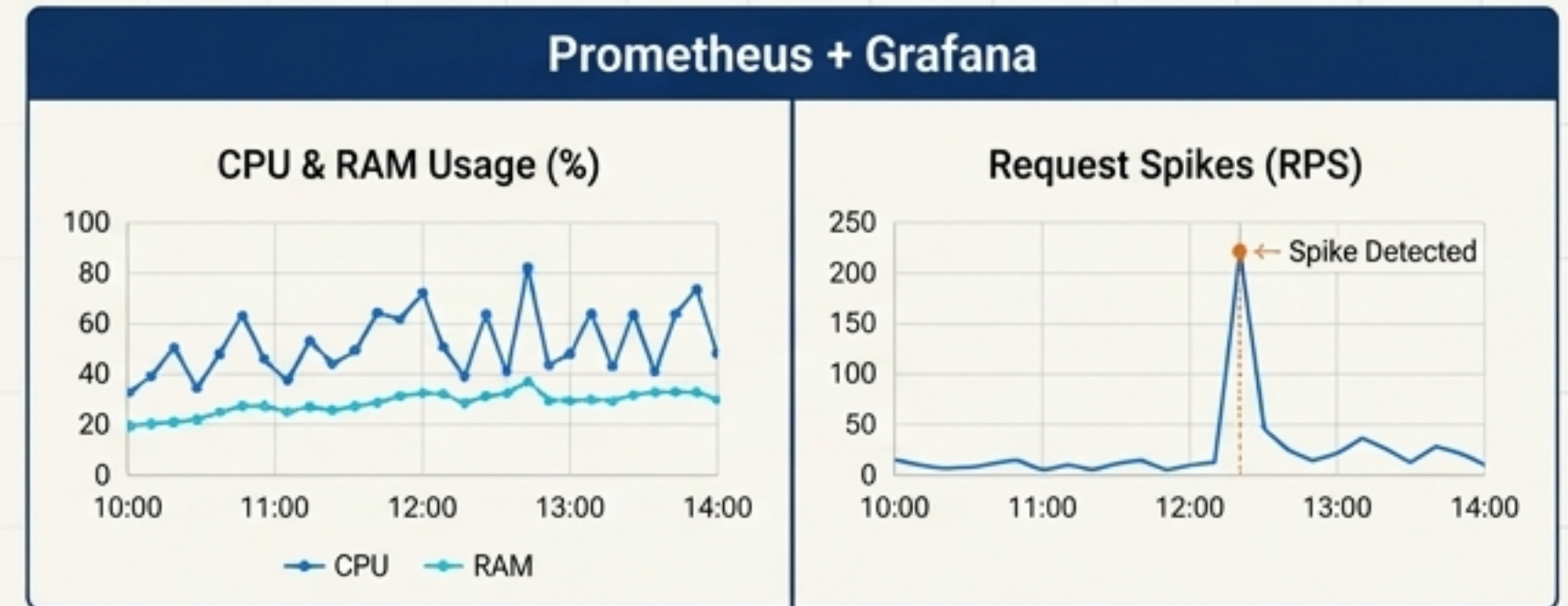
Không đo lường CPU/RAM, không phân tích được điểm nghẽn request.

[METRICS]

Prometheus + Grafana: Thu thập chỉ số (time-series) theo thời gian thực qua Spring Boot Actuator. Thiết lập biểu đồ tải và cảnh báo tự động.

[TRACING]

Jaeger: Gắn Trace IDs để lập bản đồ chi tiết từng "mắt xích" của request qua mạng lưới, xác định chính xác dịch vụ gây trễ.



Lộ Trình Chuyển Đổi: Hướng Tới Nền Tảng Microservices v2.0

[GIAI ĐOẠN 1]

Ổn Định (Stability)

Vá lỗi cấu trúc. Triển khai **Circuit Breakers (Resilience4j)**, gỡ bỏ **Hardcode**, xây dựng **Global Exception Handlers**.

[GIAI ĐOẠN 2]

An Ninh & Quan Sát

Kiến tạo **Zero Trust** nội bộ với **mTLS**. Xóa mù vận hành bằng **ELK, Prometheus, Jaeger**.

[GIAI ĐOẠN 3]

Mở Rộng & Tự Động

Chuyển đổi triệt để sang **Event-driven (Kafka)**, áp dụng **Testcontainers**, hoàn thiện **CI/CD**.

Kết Luận: Chuyển đổi từ 'mô hình tính năng' sang hệ thống enterprise thực thụ: Khả năng tự phục hồi, bảo mật sâu và mở rộng không giới hạn.